

```
"""
Tic-Tac-Toe AI (Minimax +
Alpha-Beta Pruning)
-----
-----
An unbeatable AI opponent for
Tic-Tac-Toe. The human plays
'X',
the AI plays 'O'. The AI
always picks the optimal
move, so the
best a human can achieve is a
draw.

Run it with:
    python tictactoe_ai.py
"""

import math

HUMAN, AI, EMPTY = "X", "O",
```

```

" "

WIN_LINES = [
    (0, 1, 2), (3, 4, 5), (6,
7, 8),    # rows
    (0, 3, 6), (1, 4, 7), (2,
5, 8),    # columns
    (0, 4, 8), (2, 4,
6),        # diagonals
]

def print_board(board):
    print()
    for r in range(3):
        row = board[r * 3:(r
+ 1) * 3]
        print(" |
".join(row))
        if r < 2:
            print("---+---+--
")
    print()

```

```
def winner(board):
    for a, b, c in WIN_LINES:
        if board[a] != EMPTY
and board[a] == board[b] ==
board[c]:
            return board[a]
    return None
```

```
def is_full(board):
    return EMPTY not in board
```

```
def available_moves(board):
    return [i for i, cell in
enumerate(board) if cell ==
EMPTY]
```

```
def minimax(board, depth,
alpha, beta, maximizing):
```

```
win = winner(board)
if win == AI:
    return 10 - depth
if win == HUMAN:
    return depth - 10
if is_full(board):
    return 0

if maximizing:
    best = -math.inf
    for move in
available_moves(board):
        board[move] = AI
        score =
minimax(board, depth + 1,
alpha, beta, False)
        board[move] =
EMPTY
        best = max(best,
score)
        alpha =
max(alpha, best)
        if beta <= alpha:
```

```
        break
    return best
else:
    best = math.inf
    for move in
available_moves(board):
        board[move] =
HUMAN
        score =
minimax(board, depth + 1,
alpha, beta, True)
        board[move] =
EMPTY
        best = min(best,
score)
        beta = min(beta,
best)
        if beta <= alpha:
            break
    return best

def best_move(board):
```

```

        """Return the index of
the AI's optimal move."""
        best_score = -math.inf
        move_choice = None
        for move in
available_moves(board):
            board[move] = AI
            score =
minimax(board, 0, -math.inf,
math.inf, False)
            board[move] = EMPTY
            if score >
best_score:
                best_score =
score
                move_choice =
move
        return move_choice

def human_move(board):
    while True:
        try:

```

```
        choice =
int(input("Your move (1-9):
")) - 1
        except ValueError:
            print("Please
enter a number between 1 and
9.")

            continue
        if choice not in
range(9) or board[choice] !=
EMPTY:

            print("Invalid
move, try again.")
            continue
        return choice

def main():
    board = [EMPTY] * 9
    print("Tic-Tac-Toe: You
are 'X', the AI is 'O'.")
    print("Board positions:")
    print_board([str(i + 1)
```

```
for i in range(9)])

    current = HUMAN # human
    goes first
    while True:
        if current == HUMAN:
            move =
human_move(board)
            board[move] =
HUMAN
        else:
            print("AI is
            thinking...")
            move =
best_move(board)
            board[move] = AI

    print_board(board)

    win = winner(board)
    if win:
        print(f"{win}
wins!")
```

```
        break
    if is_full(board):
        print("It's a
draw!")
        break

    current = AI if
current == HUMAN else HUMAN

if __name__ == "__main__":
    main()
```